上节课内容回顾

- 延迟分支（Delayed Branching）
  - 包括有 squash和无 squash 两种方式
- 分支预测（Branch Prediction）
  - 降低错误预测代价（即分支解析延迟）
  - 分支目标缓冲器（BTB）
- 静态分支预测（Static Branch Prediction）
- 动态分支预测（Dynamic Branch Prediction）
- 分支问题到底有多严重？（How Big Is the Branch Problem?）

回顾：更复杂的分支方向预测

- **编译期（静态）预测**
  - 始终预测为不跳转（not taken）
  - 始终预测为跳转（taken）
  - BTFN（后向分支预测跳转，前向分支预测不跳转）
    Backward taken, forward not taken
  - 基于性能分析的预测（profile based，可能的方向）
  - 基于程序分析的预测（program analysis based，可能的方向）

- **运行期（动态）预测**
  - 基于上一次结果的预测（single-bit）
    单比特方向预测器
  - 基于两比特计数器的预测（two-bit counter based prediction）
  - 两级预测（two-level prediction：全局 vs. 局部）
  - 混合预测器（Hybrid）

分支问题的重要性

- **假设条件：**
  - N = 20（20 级流水线 → 分支解析延迟为 20 周期）
  - W = 5（每周期取 5 条指令，5 发射宽度）
  - 每 5 条指令中有 1 条是分支
  - 每个 5 指令块的最后一条为分支
- **问题：取 500 条指令需要多少周期？**
  - 100% 的分支预测准确率
    - 100 周期（所有指令从正确路径取指）
    - 无任何浪费
  - 99% 的分支预测准确率
    - 100（正确路径）+ 20（一次错误路径代价）= **120 周期**
    - 20% 额外指令被取错路径
  - 98% 的分支预测准确率
    - 100（正确路径）+ 20 × 2（两次错误）= **140 周期**
    - 40% 额外指令被取错路径
  - 95% 的分支预测准确率
    - 100（正确路径）+ 20 × 5（五次错误）= **200 周期**
    - 取错路径的指令量 = 100% 的额外指令

我们还能做得更好吗？

- 上次结果预测器（Last-time）与两比特计数器预测器（2BC）利用的是“上次执行结果”的可预测性

- 观察1：一个分支的结果可能与其他分支的结果相关
  - 全局分支相关性（Global branch correlation）

- 观察2：一个分支的结果可能与该分支过去的历史结果相关（不仅仅依赖该分支上次执行时的结果）
  - 局部分支相关性（Local branch correlation）

全局分支相关性（I）

- 在执行路径中，最近执行过的分支结果与下一个分支的结果具有相关性

```
if (cond1)
...
if (cond1 AND cond2)
```

- 如果第一个分支不跳转，则第二个分支也不会跳转

```
branch Y: if (cond1) a = 2;
...
branch X: if (a == 0)
```

- 如果第一个分支跳转，则第二个分支一定不会跳转

全局分支相关性（II）

```
branch Y: if (cond1)
...
branch Z: if (cond2)
...
branch X: if (cond1 AND cond2)
```

- 如果 Y 和 Z 都跳转，则 X 也会跳转
- 如果 Y 或 Z 不跳转，则 X 也不会跳转

全局分支相关性（III）

- Eqntott，SPEC 1992 基准程序

```
if (aa==2)                    ;; B1
        aa=0;
if (bb==2)                    ;; B2
        bb=0;
if (aa!=bb) {                 ;; B3
        ....
  }
```

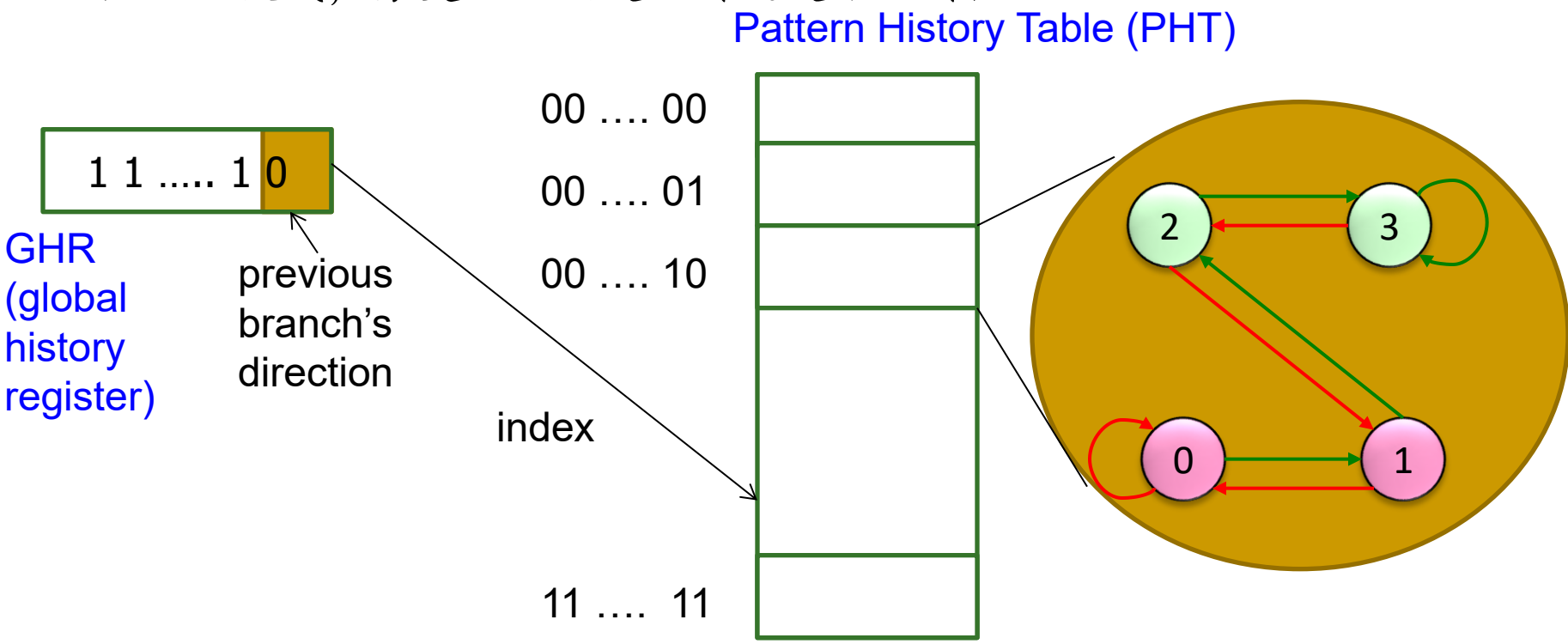
- 如果 B1 不跳转（即 aa 在执行 B3 时为 0），并且 B2 不跳转（即 bb 在执行 B3 时为 0），那么 B3 就一定会跳转

捕获全局分支相关性

- **核心思想：将分支结果与所有分支的“全局跳转/不跳转历史（global T/NT history）”关联起来。**
- **基于全局历史进行预测：根据上一次出现相同全局分支历史时该分支的执行结果来进行预测。**
- **实现方法：**
  - 维护所有分支的“全局跳转/不跳转历史”，存放在一个寄存器中 → 全局历史寄存器（Global History Register，GHR）
  - 使用 GHR 作为索引访问一张表，该表记录了最近一段时间中每个 GHR 模式对应的分支结果 →
    模式历史表（Pattern History Table，PHT），通常是由 2-bit 计数器组成的表
- **全局历史 / 分支预测器（global history/branch predictor）**
- **使用两级历史信息：**
  - GHR（全局历史）
  - 在该 GHR 下记录的分支行为历史（PHT 记录）

两级全局分支预测

- 第一级：全局分支历史寄存器（GHR，N 位）
  - 记录最近 N 个分支的跳转方向（跳转 = 1，不跳转 = 0）
- 第二级：饱和计数器表（每个历史项对应一个）
  - 记录在上一次出现相同的历史模式时，该分支的实际走向（即：根据同样的 GHR 模式，分支上一次是跳转还是不跳转）



Yeh and Patt, "Two-Level Adaptive Training Branch Prediction," MICRO 1991.

全局预测器是如何工作的？

```
for (i = 0; i < 100; i++)
    for (j = 0; j < 3; j++)
```

在初始启动阶段结束后，条件分支表现如下（假设全局寄存器 GR 采用左移方式更新）：

测试条件 test	变量值 value	GR (全局历史)	分支结果 result
j < 3	j = 1	1101	跳转 (taken)
j < 3	j = 2	1011	跳转 (taken)
j < 3	j = 3	0111	不跳转 (not taken)
i<100	——	1110	通常跳转 (usually taken)

- 该分支测试变量 i
- 最近 4 次测试 j 的分支结果分别为：
TTTN（跳转、跳转、跳转、不跳转）
- 根据历史 TTTN，对 i 进行分支预测：预测为跳转
- 下一个历史将变为：TTNT（将最新结果左移进入 GR）

- McFarling, *"Combining Branch Predictors,"* DEC WRL TR, 1993.

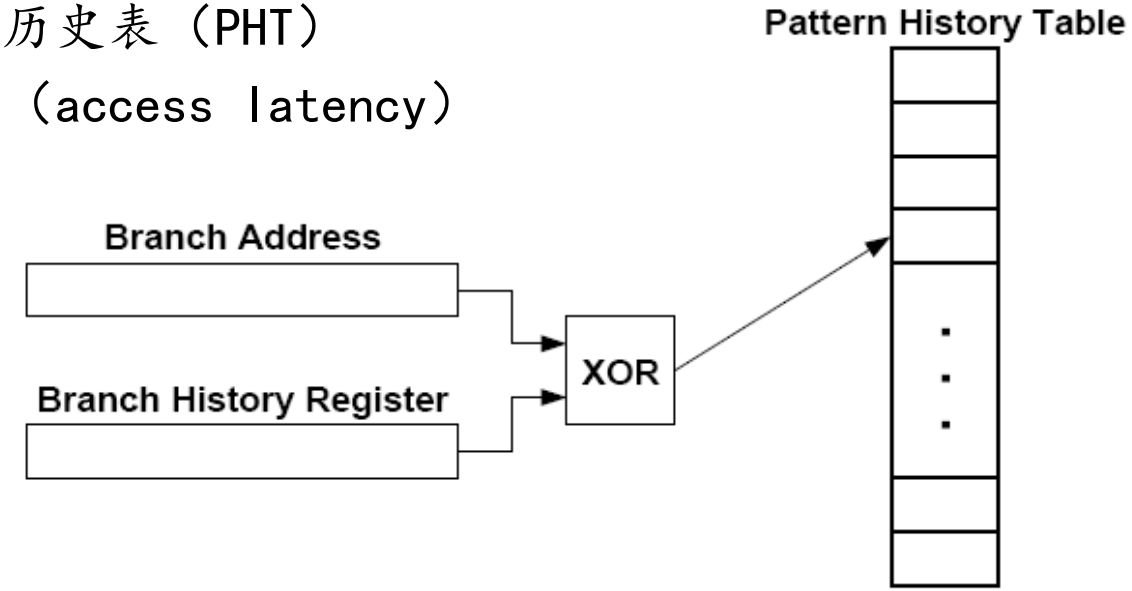
Intel Pentium Pro 分支预测器

- 4 位全局历史寄存器（GHR）
- 多个模式历史表（Pattern History Tables），每个由 2 位饱和计数器组成
  - 使用哪一张模式历史表由分支指令地址的低位比特决定

- 预测流程大致是：
  1. 从分支地址的低位比特决定使用哪一张 PHT
  2. 从 GHR决定在这张 PHT 中查哪个条目
  3. 得到对应的 2-bit 饱和计数器
  4. 根据计数器状态预测该分支是否跳转

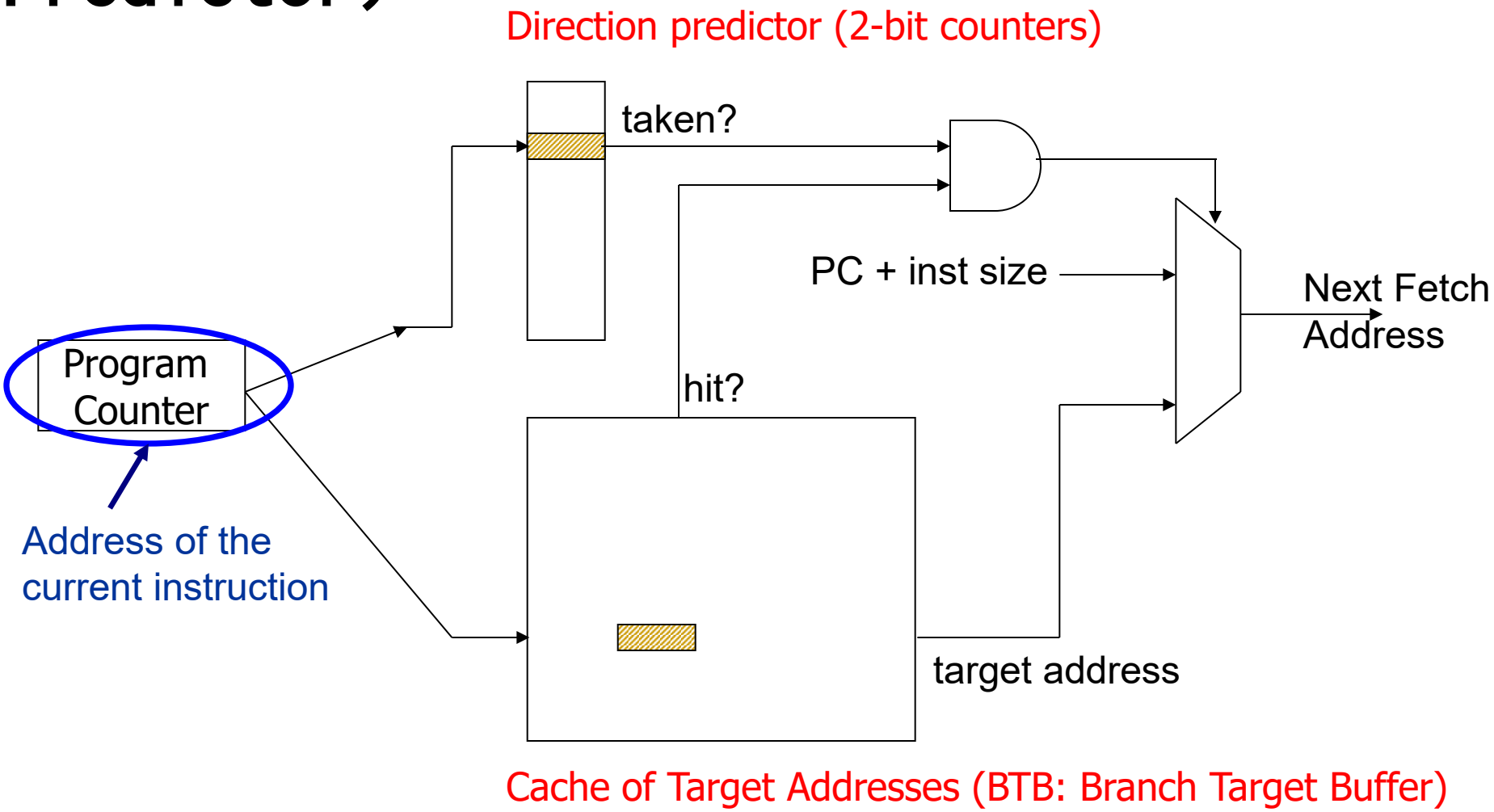
提升全局预测器的准确率

- **核心思想：在全局预测器中加入更多上下文信息，以便区分正在被预测的是哪条分支指令。**
  - Gshare 分支预测器：将 GHR（全局历史寄存器） 与 分支指令的 PC 地址 进行 XOR（异或）混合得到索引
  - +提供更多上下文信息
  - +更高效地利用模式历史表（PHT）
  - - 增加了访问延迟（access latency）

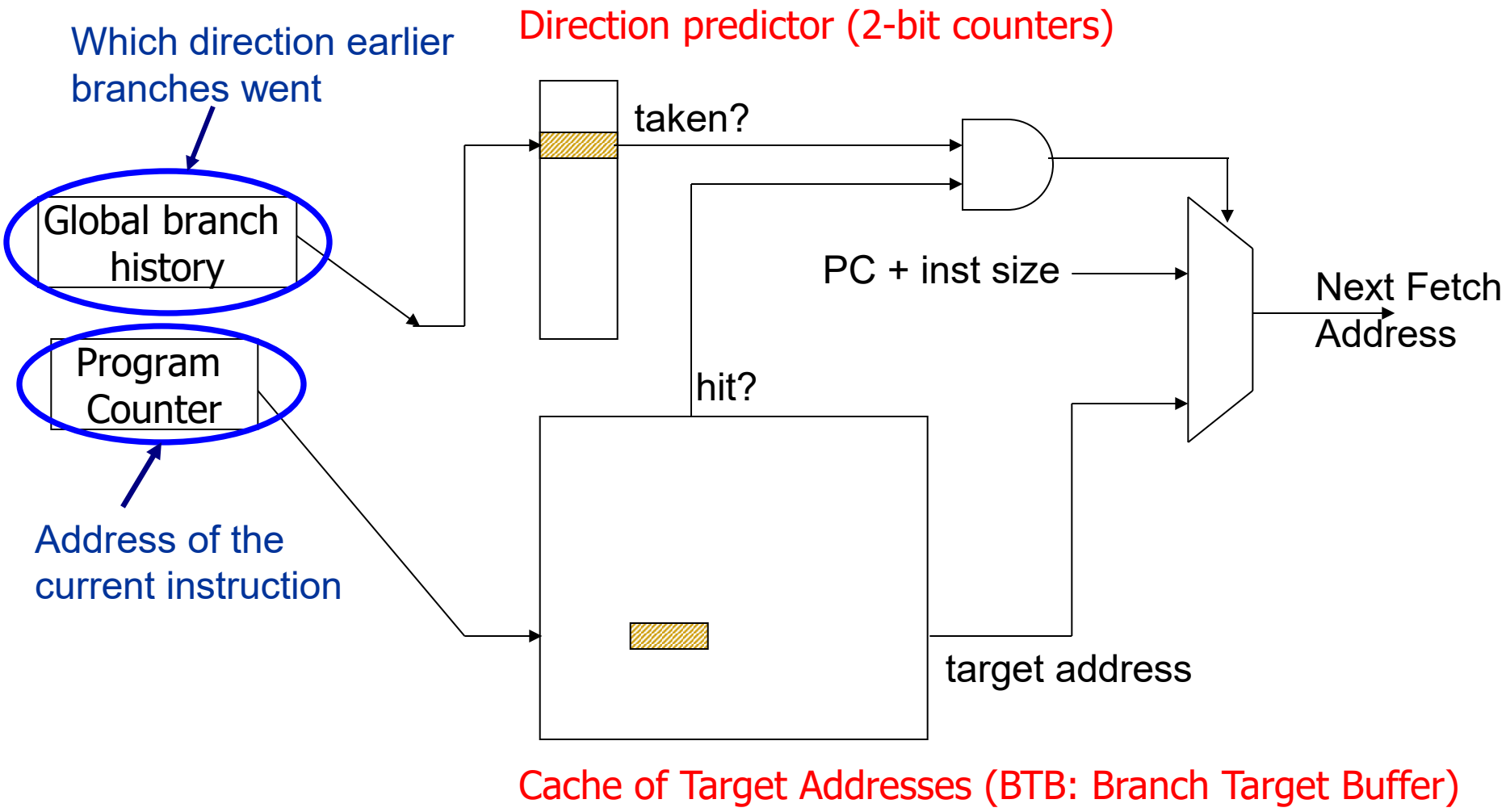


McFarling, “Combining Branch Predictors,” DEC WRL Tech Report, 1993.

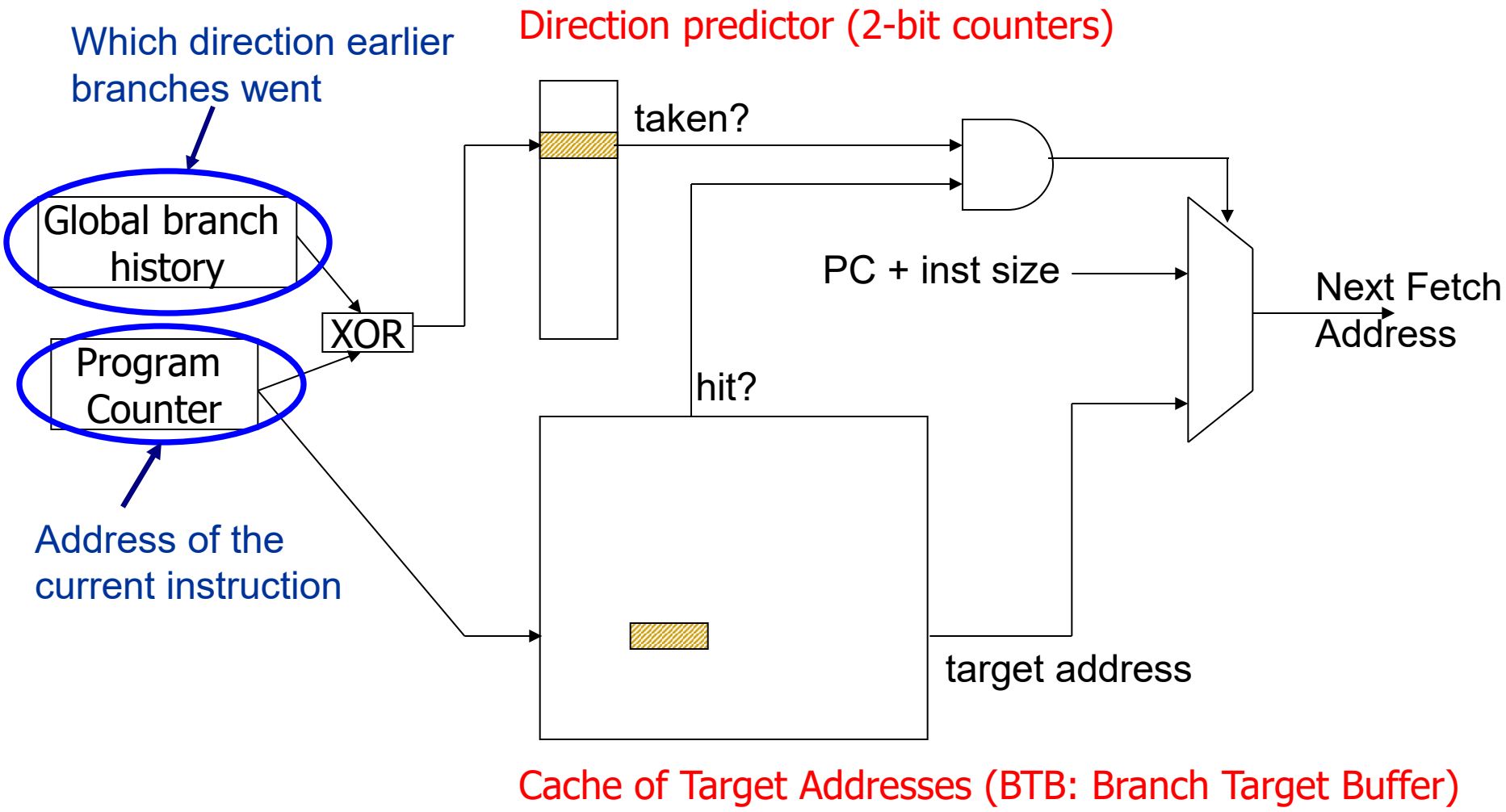
回顾：一级分支预测器（One-Level Branch Predictor）



两级全局历史分支预测器（Two-Level Global History Branch Predictor）



两级 Gshare 分支预测器（Two-Level Gshare Branch Predictor）



我们还能做得更好吗？

- 上次结果预测器（Last-time）与两比特计数器预测器（2BC）利用的是“上次执行结果”的可预测性

- 观察1：一个分支的结果可能与其他分支的结果相关
  - 全局分支相关性（Global branch correlation）

- 观察2：一个分支的结果可能与该分支过去的历史结果相关（不仅仅依赖该分支上次执行时的结果）
  - 局部分支相关性（Local branch correlation）

局部分支相关性

```
for (i=1; i<=4; i++) { }
```

If the loop test is done at the end of the body, the corresponding branch will execute the pattern $(1110)^n$, where 1 and 0 represent taken and not taken respectively, and n is the number of times the loop is executed. Clearly, if we knew the direction this branch had gone on the previous three executions, then we could always be able to predict the next branch direction.

- McFarling, “Combining Branch Predictors,” DEC WRL TR 1993.

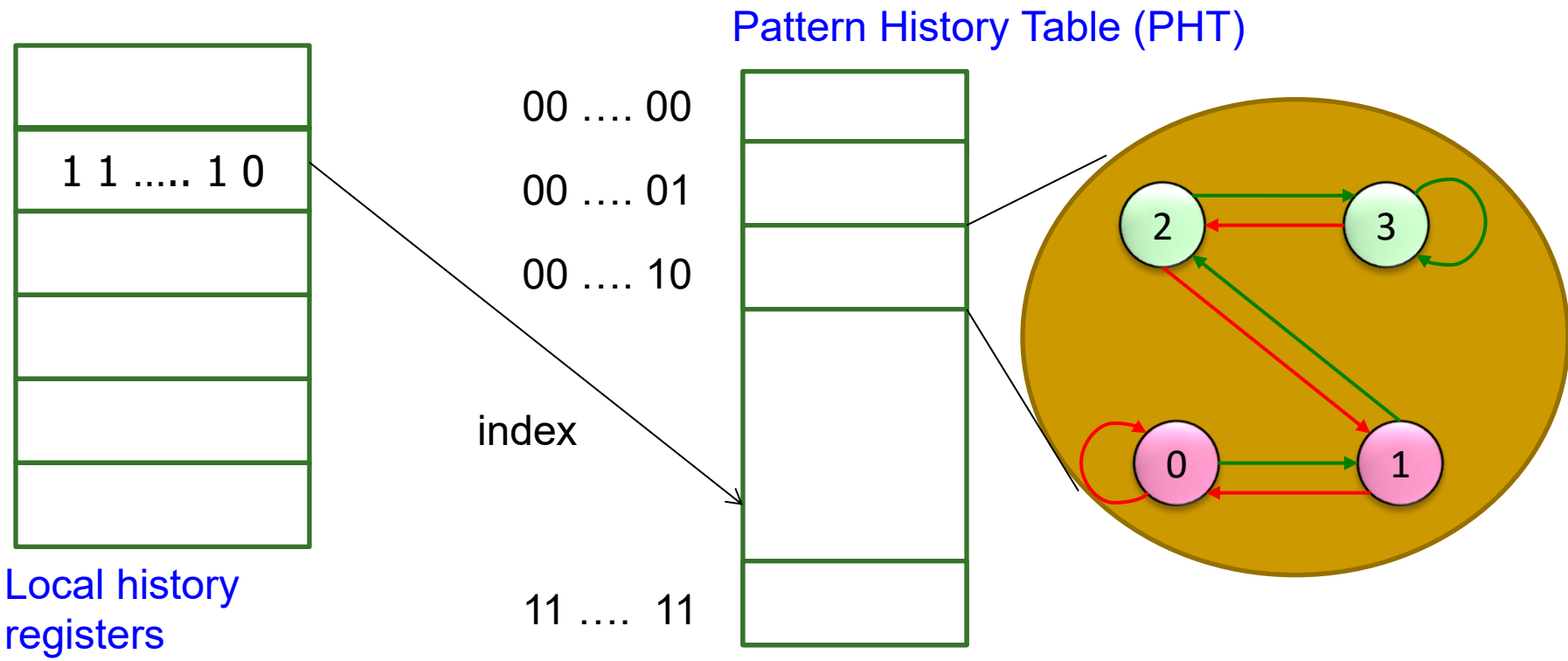
捕获局部分支相关性

- **核心思想：使用每条分支独立的历史寄存器（per-branch history register）**
  - 将分支的预测结果与该分支自身的 “跳转/不跳转（T/NT）历史” 相关联，当再次遇到相同的局部分支历史模式时
    → 基于上次该模式出现时该分支的实际结果进行预测

- **局部历史预测器（local history / branch predictor）**
  - 使用两级历史信息：
    - **每条分支的历史寄存器（per-branch history register）**
    - **对应该历史模式下的分支行为记录（history at that register value）**

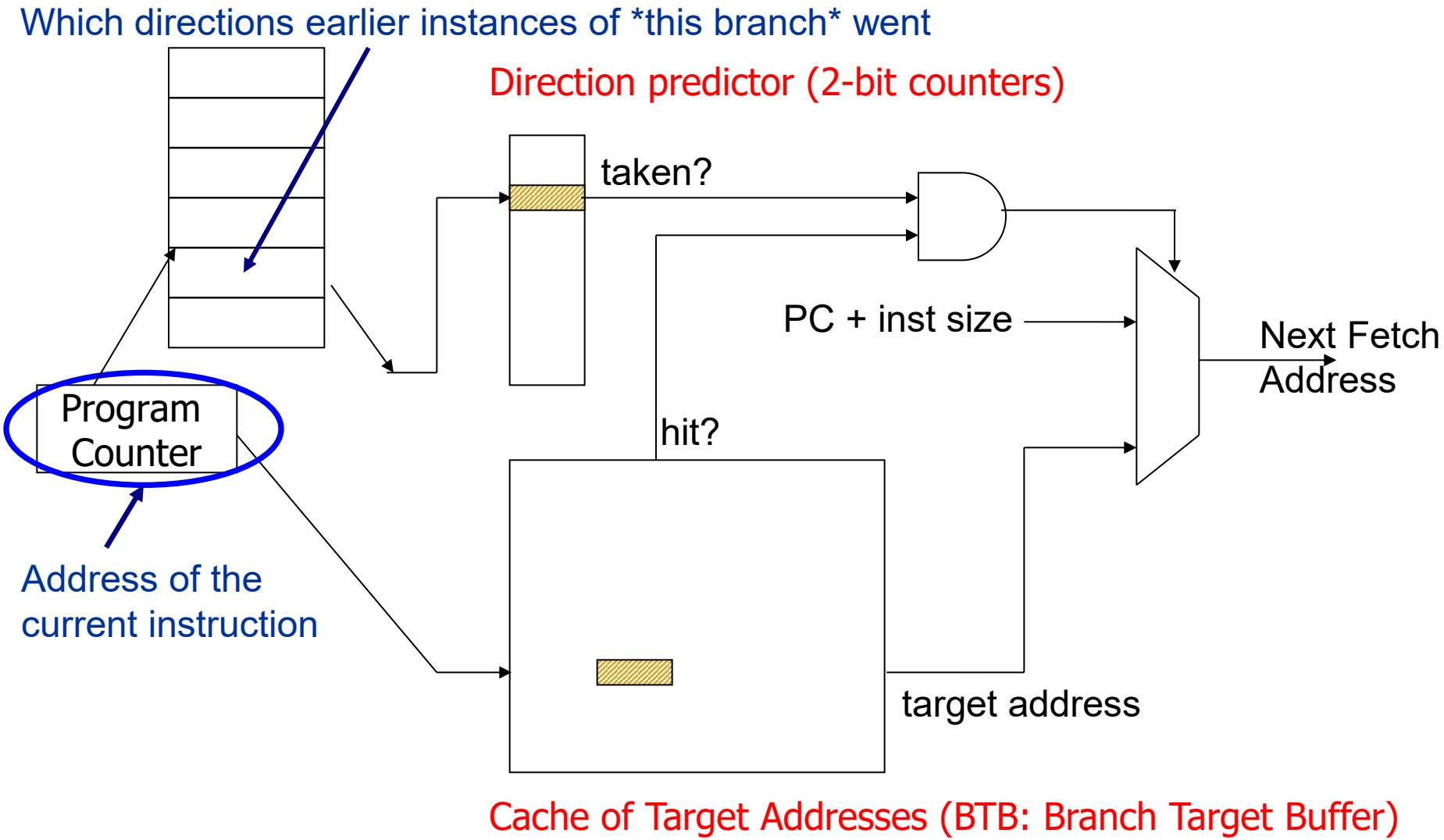
两级局部分支预测

- **第一级：一组局部分支历史寄存器（每个 N 位）**
  - 为每条分支分配一个局部历史寄存器
  - 根据分支指令的 PC 地址 选择要使用的历史寄存器
- **第二级：饱和计数器表（针对每个历史模式）**
  - 每个局部历史模式对应一个 2 位饱和计数器



Yeh and Patt, "Two-Level Adaptive Training Branch Prediction," MICRO 1991.

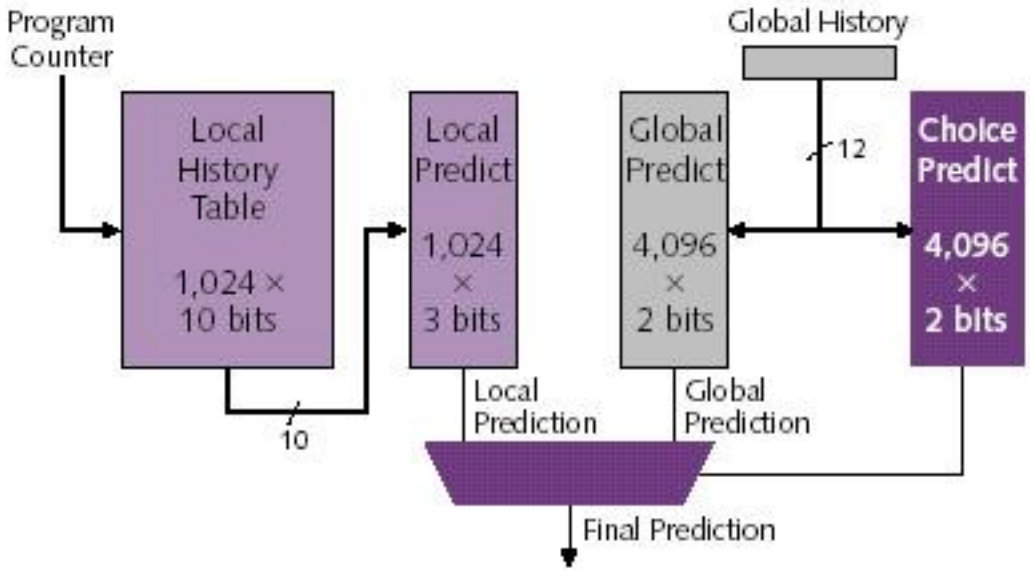
两级局部历史分支预测器



混合分支预测器

- **核心思想：使用 多种不同类型的预测器（即多个算法），并从中选择 “最佳预测”。**
  - 示例：将 2 位饱和计数器预测器 与 全局预测器 组合构成混合预测器。
- **优点：**
  - 准确率更高，不同类型的预测器适合不同类型的分支，因此组合后预测更稳定。
  - 更短的预热时间（warmup time）可以在慢预测器预热期间，先使用预热更快的预测器。
- **缺点：**
  - 需要额外的“元预测器（meta-predictor）”或“选择器（selector）”用于决定应该相信哪个预测器。
  - 更长的访问延迟（access latency）因为需要查询多个预测器并做选择判断。
- McFarling, *“Combining Branch Predictors,”* DEC WRL Tech Report, 1993.

Alpha 21264 锦标赛（Tournament）分支预测器



- 最小分支开销（minimum branch penalty）：7 个周期（cycles）
- 典型分支开销（typical branch penalty）：11 个以上周期
- 48K bits 的目标地址缓存在指令缓存（I-cache）中
- 在上下文切换（context switch）时，预测器表会被重置（reset）

- Kessler, *“The Alpha 21264 Microprocessor,”* IEEE Micro, 1999.

分支预测准确率（示例）

- Bimodal：使用分支地址索引的 2-bit 饱和计数器表
  - 浅灰色：Bimodal 预测器中灰色：Gshare 预测器黑色：Bimodal/Gshare 混合预测器

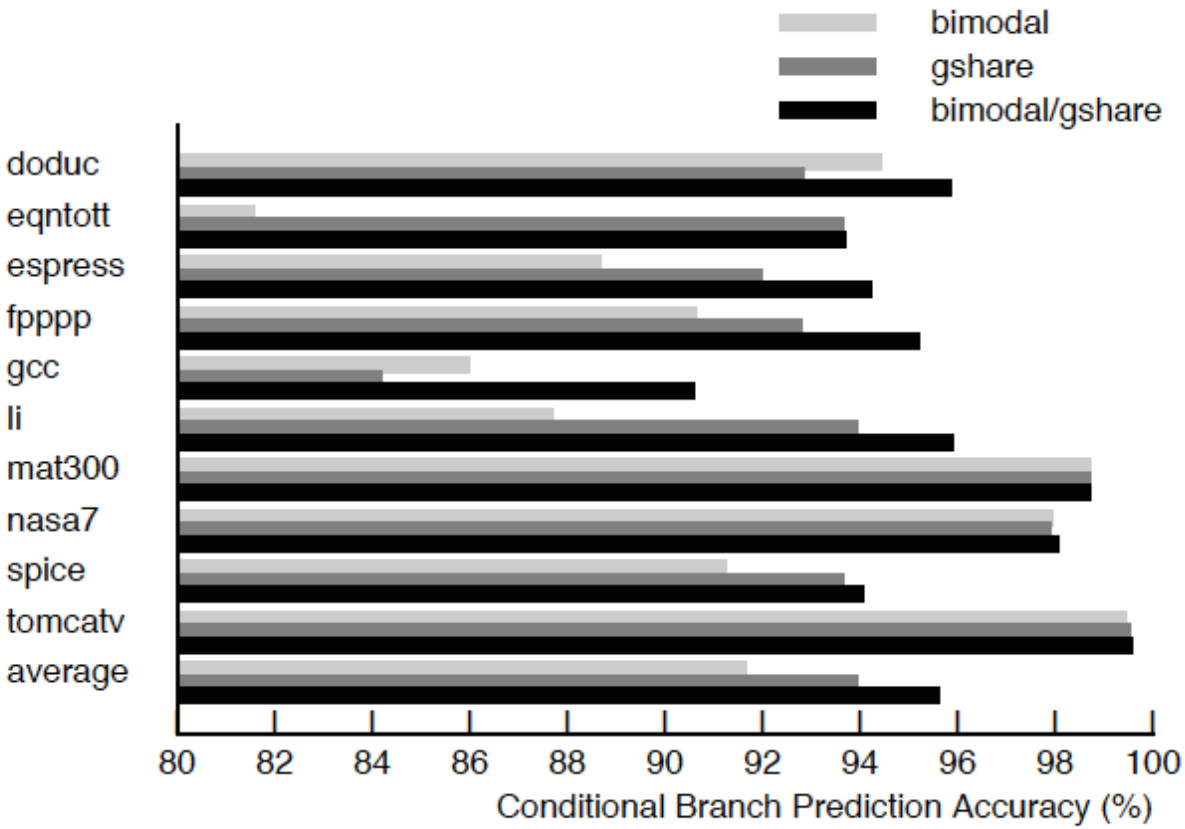


Figure 13: Combined Predictor Performance by Benchmark

偏置分支（Biased Branches）

- 观察（Observation）：许多分支在方向上具有严重偏置，例如：99% 都是跳转（taken）

- 问题（Problem）：这些偏置分支会 污染（pollute）分支预测结构，导致以下后果：
  - 在分支预测表（如 PHT）中引入不必要的更新，在历史寄存器中造成“干扰（interference）”，使其他真正困难的分支更难预测

- 解决方法（Solution）：检测这些偏置分支，并用更简单的预测器处理它们，例如：last-time predictor（上次结果预测器），静态预测器（static predictor），或其他简单策略

  - Chang et al., *“Branch classification: a new mechanism for improving branch predictor performance,”* MICRO 1994.

其他一些分支预测器类型

- 循环分支检测器与预测器（Loop branch detector and predictor）
  - 特别适用于迭代次数较少且迭代次数可预测的循环
- 感知器（Perceptron）分支预测器
  - 学习不同分支之间的方向相关性（direction correlations）
  - 为不同相关性分配权重（assigns weights）
  - Jimenez & Lin, *“Dynamic Branch Prediction with Perceptrons,”* HPCA 2001.
  - （注：这是引入机器学习思想的第一类分支预测器）
- 几何历史长度预测器（Geometric history length predictor）
  - （使用呈几何级数的不同历史长度，属于高精度现代预测器家族，如 TAGE 系列的前身）

回顾：如何处理控制依赖

- 保持流水线完整，确保动态指令的正确顺序是至关重要的。

- 如果指令是控制流指令，潜在的解决方案有：

- 暂停流水线，直到我们知道下一个取址地址。
- 猜测下一个取址地址（分支预测）。
- 使用延迟分支（分支延迟槽）。
- 做其他处理（细粒度多线程）。
- 消除控制流指令（谓词执行 predicated execution）。
- 从两个可能的路径中取指（如果你知道两个路径的地址）（多路径执行）。

回顾：谓词合并（而非谓词执行）

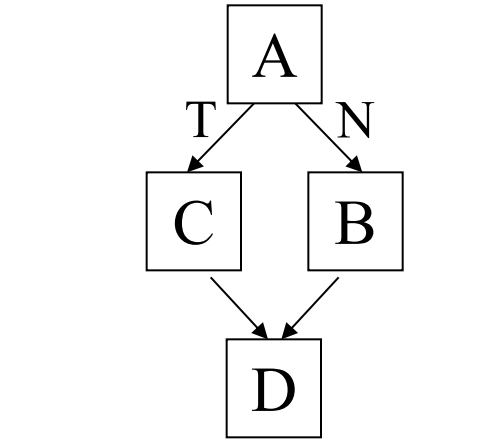
- **复杂的谓词(Predicates)会被转换成多个分支：**
  - 例如，如果((a == b) && (c < d) && (a > 5000)) { … }
  - 这会有3个条件分支
- **问题：这会增加控制依赖的数量**
- **想法：合并谓词操作，将结果传递给一个单一的分支指令，而不是为每个谓词生成一个分支**
  - 谓词通过条件寄存器进行存储和操作
  - 单个分支指令检查合并后谓词的值
  - 代码中分支较少 → 较少的误预测/停顿
  - 可能的多余工作
  - 如果第一个谓词为假，则无需计算其他谓词
- **条件寄存器存在于IBM RS6000和POWER架构中**

谓词化执行（Predicated Execution）

- **核心思想：编译器将控制依赖（control dependence）转换为数据依赖（data dependence），从而 消除分支（branch is eliminated）。**
  - 每条指令根据谓词计算结果带有一个 谓词位（predicate bit）
  - 只有谓词为 TRUE 的指令才会真正执行和提交（commit）其余谓词为 FALSE 的指令会变成 NOP（空操作）

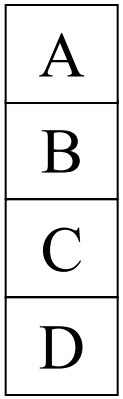
```
if (cond) {
    b = 0;
}
else {
    b = 1;
}
```

(normal branch code)




```
A   p1 = (cond)
    branch p1, TARGET
B   mov b, 1
    jmp JOIN
C   TARGET:
    mov b, 0
D   add  x, b, 1
```

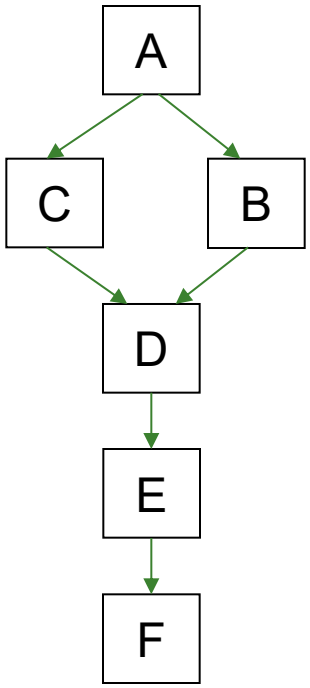
(predicated code)




```
A   p1 = (cond)
B   (!p1) mov  b, 1
C   (p1) mov  b, 0
D   add  x, b, 1
```

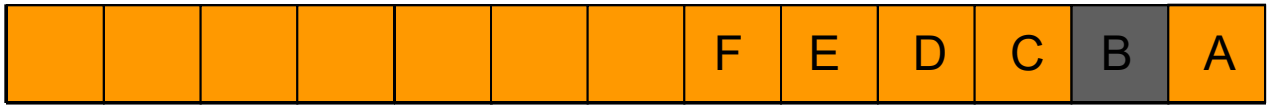
谓词执行（II）

- 谓词执行可以同时实现高性能与高能效



Predicated Execution

Fetch Decode Rename Schedule RegisterRead Execute



nop

Branch Prediction

Fetch Decode Rename Schedule RegisterRead Execute



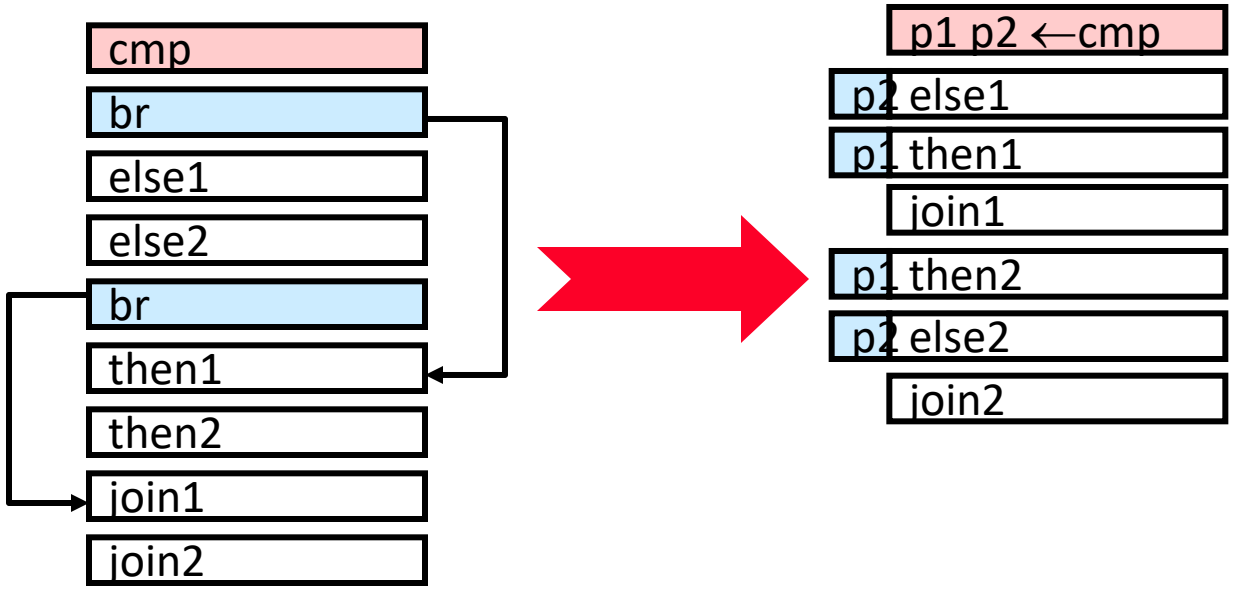
Pipeline flush!!

谓词执行（III）

- **优势（Advantages）：**
  - 消除难以预测分支的误预测开销
    - 某些分支无需使用分支预测器
    - 当误预测开销 > 谓词执行造成的额外无效工作量时，谓词化是有利的
  - 允许更多代码优化，减少来自控制依赖的限制
    - 在谓词化代码中可以更自由地移动指令
- **劣势（Disadvantages）：**
  - 对容易预测的分支会造成额外无效工作
    - 如果 误预测代价 < 谓词化带来的额外工作量，性能会下降（Reduces performance if misprediction cost < useless work）
    - 适应性问题（Adaptivity）：静态谓词化无法适应运行时的分支行为变化（行为会随着输入集、程序阶段、控制流路径等动态变化）
  - 需要额外的硬件和 ISA 支持，如谓词寄存器、谓词位、谓词编码等架构扩展
  - 无法消除所有难预测分支，特别是循环类分支（loop branches）仍然可能难以预测

Intel Itanium中的谓词执行

- 每条指令都可以独立进行谓词化
- Itanium 提供 64 个 1-bit 谓词寄存器（predicate registers）
  - 每条指令携带 6 位谓词字段（predicate field）（用于指定该指令依赖哪一个谓词寄存器）
- 如果谓词为 FALSE，该指令等效于 NOP



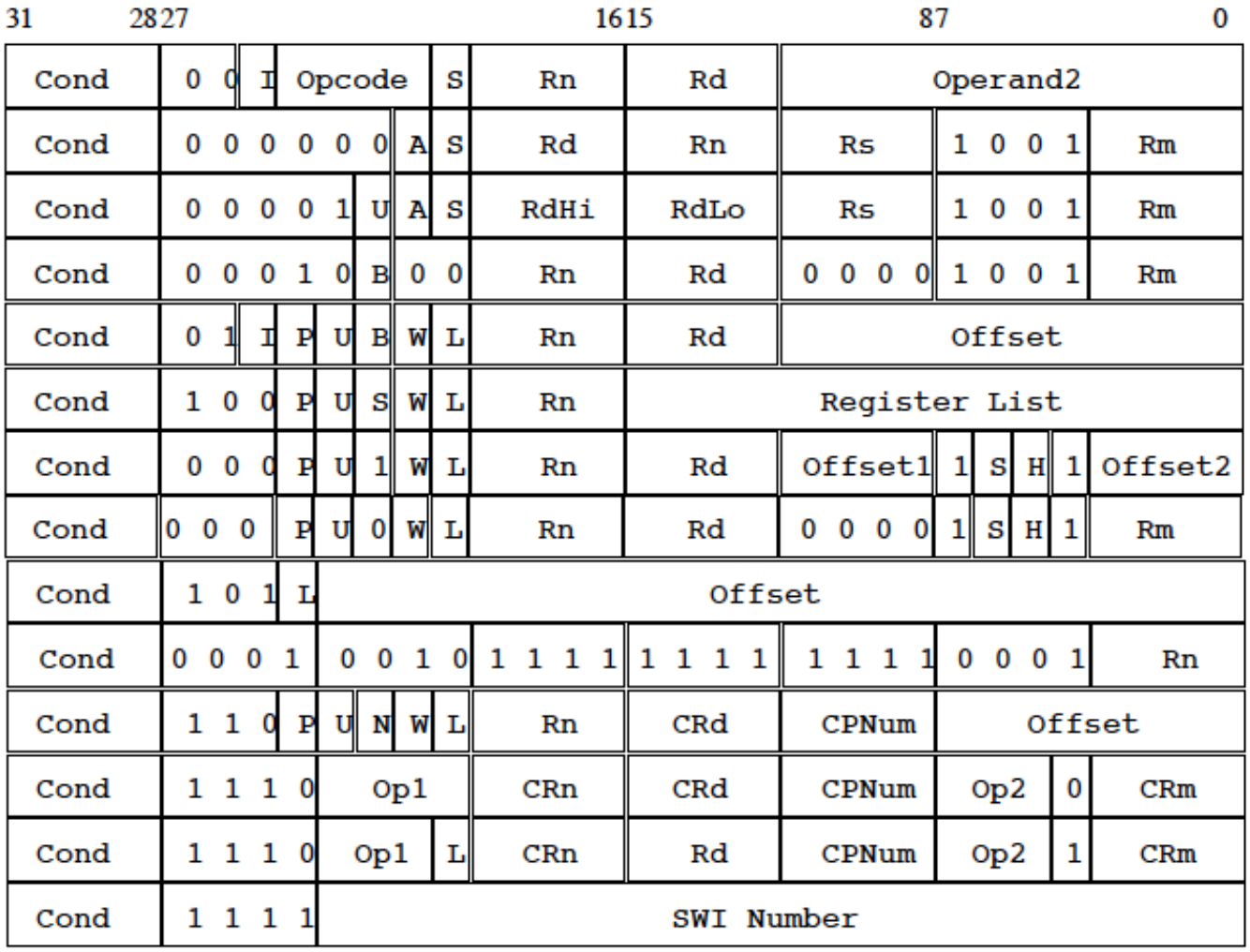
ARM 指令集中的条件执行

- ARM 指令集中的条件执行（Conditional Execution in the ARM ISA）

- 几乎所有 ARM 指令都可以带一个可选的条件码（condition code）

- 带条件码的指令只有在 CPSR 中的条件码标志满足条件时才会执行
  - CPSR（Current Program Status Register）中包含 NZCV 等条件标志
  - 指令如 ADDNE, MOVEQ, SUBGT 等仅在条件成立时执行
  - 条件不满足时，该指令被当作 NOP（不改变状态、不产生效果）

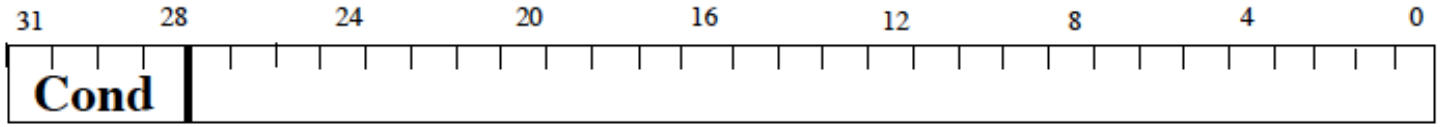
ARM 指令集中的条件执行



Instruction type

- Data processing / PSR Transfer
- Multiply
- Long Multiply **(v3M / v4 only)**
- Swap
- Load/Store Byte/Word
- Load/Store Multiple
- Halfword transfer : Immediate offset **(v4 only)**
- Halfword transfer: Register offset **(v4 only)**
- Branch
- Branch Exchange **(v4T only)**
- Coprocessor data transfer
- Coprocessor data operation
- Coprocessor register transfer
- Software interrupt

ARM 指令集中的条件执行



0000 = EQ - Z set (equal)

0001 = NE - Z clear (not equal)

0010 = HS / CS - C set (unsigned higher or same)

0011 = LO / CC - C clear (unsigned lower)

0100 = MI -N set (negative)

0101 = PL - N clear (positive or zero)

0110 = VS - V set (overflow)

0111 = VC - V clear (no overflow)

1000 = HI - C set and Z clear (unsigned higher)

1001 = LS - C clear or Z (set unsigned lower or same)

1010 = GE - N set and V set, or N clear and V clear (>or =)

1011 = LT - N set and V clear, or N clear and V set (>)

1100 = GT - Z clear, and either N set and V set, or N clear and V set (>)

1101 = LE - Z set, or N set and V clear,or N clear and V set (<, or =)

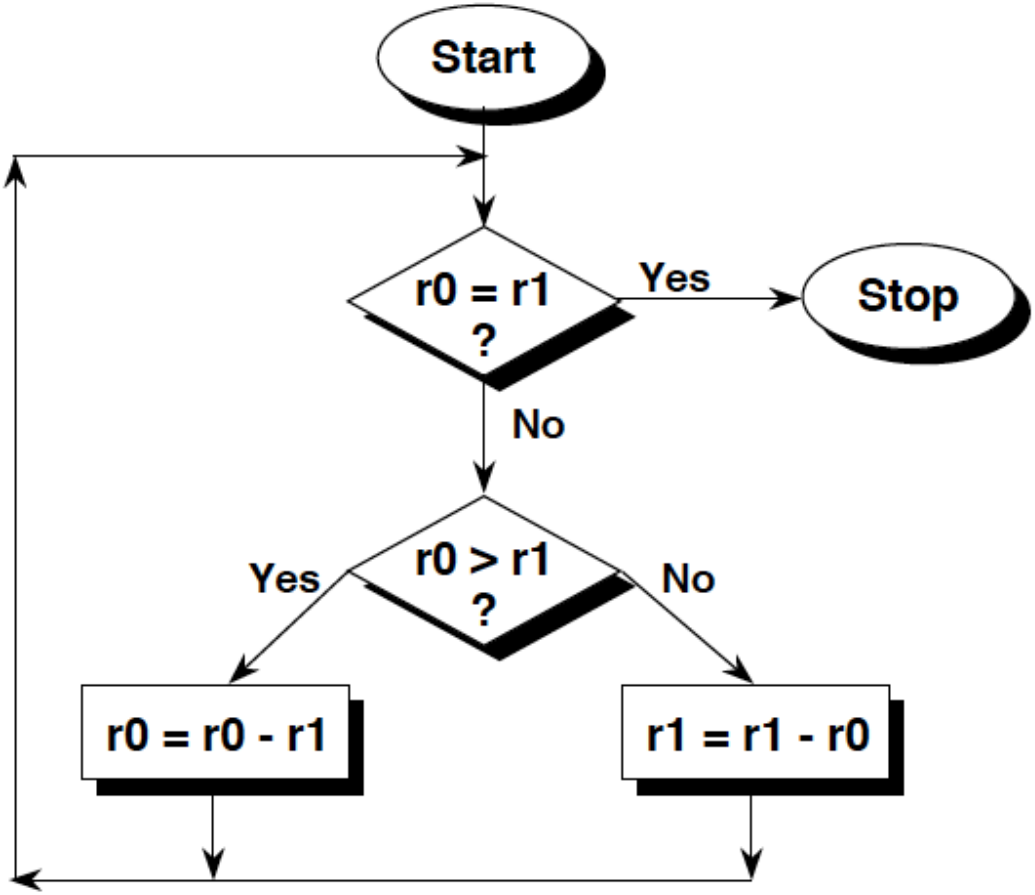
1110 = AL - always

1111 = NV - reserved.

ARM 指令集中的条件执行

* **To execute an instruction conditionally, simply postfix it with the appropriate condition:**
  - For example an add instruction takes the form:
    - `ADD r0,r1,r2 ; r0 = r1 + r2 (ADDAL)`
  - To execute this only if the zero flag is set:
    - `ADDEQ r0,r1,r2 ; If zero flag set then...`
      `; ... r0 = r1 + r2`
* **By default, data processing operations do not affect the condition flags (apart from the comparisons where this is the only effect). To cause the condition flags to be updated, the S bit of the instruction needs to be set by postfixing the instruction (and any condition code) with an "S".**
  - For example to add two numbers and set the condition flags:
    - `ADDS r0,r1,r2 ; r0 = r1 + r2`
      `; ... and set flags`

ARM 指令集中的条件执行



* **Convert the GCD algorithm given in this flowchart into**
  1) "Normal" assembler, where only branches can be conditional.
  2) ARM assembler, where all instructions are conditional, thus improving code density.

* **The only instructions you need are CMP, B and SUB.**

ARM 指令集中的条件执行

"Normal" Assembler

```
gcd     cmp r0, r1        ;reached the end?
        beq stop
        blt less          ;if r0 > r1
        sub r0, r0, r1    ;subtract r1 from r0
        bal gcd
less    sub r1, r1, r0    ;subtract r0 from r1
        bal gcd
stop
```

ARM Conditional Assembler

```
gcd     cmp   r0, r1      ;if r0 > r1
        subgt r0, r0, r1  ;subtract r1 from r0
        sublt r1, r1, r0  ;else subtract r0 from r1
        bne   gcd         ;reached the end?
```

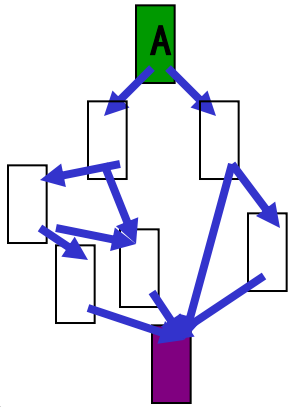
理想化（Idealism）

- **要是能够**
  - 只有在分支实际上会被误预测时，才将其消除（采用谓词化）
  - 只有在分支实际上会被正确预测时，才对其进行预测

- **要是能够**
  - 如果谓词化不需要任何 ISA（指令集架构）支持就好了

改进谓词执行

- **谓词化的三大主要局限：**
  - 适应性（Adaptivity）不足：
    - 无法适应运行时分支行为的动态变化，静态谓词化不能根据输入、阶段、控制流变化调整策略
  - 复杂控制流（Complex CFG）：
    - 不适用于循环（loops）或复杂控制流图（complex control flow graphs），控制路径合并点多、结构复杂，谓词化难以覆盖
  - ISA 需求大（ISA）：
    - 需要对指令集进行较大幅度修改



- **Wish Branches（Kim+, MICRO 2005）**
  - **解决局限 1（适应性）**
  - **部分解决局限 2（针对循环）**
- **动态谓词执行（Dynamic Predicated Execution）**
  - 基于 Diverge-Merge Processor（分叉-合并处理器）
    参考：Kim+, MICRO 2006
  - 解决问题：**局限 1（适应性）局限 2（部分）局限 3（部分）**